

COMENZANDO CON PYTHON: Breve guía para la programación en Python3

Introducción al lenguaje

Fue desarrollado por Guido van Rossum a finales de los 80 en el instituto de investigación holandés Centro para las Matemáticas y la Informática (CWI, Centrum Wiskunde & Informatica).

Las principales características del lenguaje son:

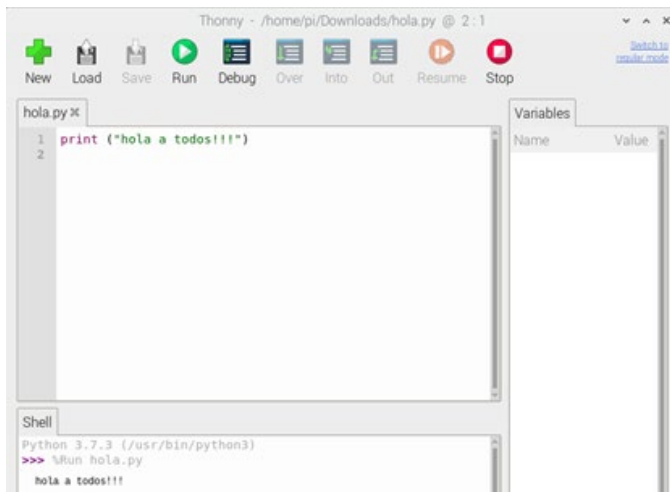
- Lenguaje de programación de alto nivel.
- Lenguaje interpretado.
- Tipado dinámico.
- Sintaxis clara y expresiva.
- Multiparadigma.
- Multiplataforma.
- Licencia de código abierto.
- Extensa biblioteca.

Python 3.0 fue lanzado en 2008. La última versión Python 2.7 se lanzó en 2010. Ambas versiones son intencionadamente incompatibles.

Muchas distribuciones de Linux instalan Python de forma predeterminada. En windows debemos instalar python. Para ello desde la [página oficial de python](#) nos descargamos la última versión que haya.

A la hora de programar tenemos dos formas de hacerlo:

- Desde la línea de comandos ejecutamos python para entrar en el intérprete. Los signos >>> es donde debemos introducir las sentencias.
- Usar un entorno de desarrollo integrado (IDE). Existen bastantes entornos pero algunos sencillos de usar son: el que nos viene por defecto con la instalación o **IDLE** o otro llamado **Thonny**.



Guía de estilo

Se debe tener en cuenta unas reglas de estilo para escribir nuestros programas:

- El código se escribe en cada línea desde el primer espacio. Cuando es necesario el sangrado se usan 4 espacios (o múltiplos).
- El tamaño de la línea es máximo de 79 caracteres.
- Comentarios, a ser posible, en una sola línea (#), si abarcan varias líneas, debemos comenzarlos y terminarlos con tres comillas (simples o dobles).
- Usar espacios alrededor de operadores y luego de comas.
- Usar líneas en blanco para separar funciones y clases.
- Se utilizan variables que permiten almacenar información para usarla más tarde.
- Se puede controlar el camino de ejecución usando bucles y sentencias de control.
- Se puede reutilizar código usando funciones, que hacen los programas más fáciles de entender y mantener.
- Se puede importar módulos y usar métodos ya definidos para realizar parte del trabajo.

Consejos Python para programar bien:

```
import this
```

Conceptos y elementos básicos de la programación: Variables, tipos y estructuras de datos, funciones, objetos.

Variables y sus tipos básicos

Un programa en su funcionamiento usará datos que pueden ser letras, números,... que irán cambiando de valor de acuerdo a las diferentes acciones que se lleven a

cabo sobre ellos. Una de las características más potentes de un lenguaje de programación es la capacidad de almacenar en memoria el valor en cada momento de un dato en concreto. Llamamos **variable** al nombre que se le asigna en el programa a un dato concreto que tendrá un valor determinado en cada momento en memoria.

En Python se utiliza una sentencia de asignación (usamos el signo 'igual', =) para crear una variable nueva dándole un valor. Por ejemplo:

```
a = 3
texto = "hola"
```

El tipo de la variable será el del dato asignado, es decir, no se declara el tipo de la variable al crearla. Si una variable no está definida (no tiene un valor asignado) al intentar usarla se producirá un error. La última expresión impresa se asigna a la variable “_”.

Los programadores generalmente eligen nombres para sus variables que tengan sentido y definan para qué se usa esa variable. Los nombres de las variables pueden contener tanto letras como números, pero no pueden comenzar con un número. Por convenio, el nombre comienza en minúscula y si son varias palabras se unen por guión bajo.

El tipo de la variable es el del dato asignado, es decir, la variable *a* es de tipo entero y la variable *texto* es de tipo cadena de caracteres o string. En otros lenguajes de programación es necesario declarar el tipo de la variable, antes de asignarle un valor. Sin embargo, en Python, se declara el tipo de la variable al crearla, es lo que se define en programación como **tipado dinámico**.

Los tipos básicos son:

- **Numérico:** que puede ser entero (int), real (float) y complejo (complex).
- **Lógicos o booleanos** (bool).
- **Cadenas de caracteres o string** (str).

Para saber el tipo de una variable usamos la expresión: `type (nombre_variable)`. Así:

```
type (texto)
```

La asignación define el tipo de variable:

```
a = 10
b = 10.1
c = 1.2e-3
d = 1.2+1.5j
```

```
type(a)
```

```
type(b)
```

```
type(c)
```

```
type(d)
```

Antes de seguir con *los tipos de variables* vamos a explicar que es un **comentario**. En los lenguajes de programación para ayudar a entender qué realiza un programa, por ejemplo, a una persona que revise el código que no sea el programador, se utilizan comentarios al lado de las sentencias, que expliquen o clarifiquen su funcionamiento. Esto se llama *documentar el programa*. Para que el intérprete de Python, entienda que es un *comentario* y no una sentencia a ejecutar, se utiliza el símbolo #.

Tipo Numérico

Para trabajar con Números tenemos los **Operadores Aritméticos**:

Operador	descripción
+	Suma
-	Resta
*	Multiplicación
**	Potencia
/	División
//	División, se obtiene parte entera del cociente
%	Módulo, se obtiene el resto

```
suma = 3 + 2 #el valor de la variable suma es 5  
suma
```

```
div = 6 / 3 #el valor de la variable div es 2.0  
div
```

También podemos trabajar con **Operadores binarios**:

Operador	descripción
&	and
	or
~	not
<<	Desplazamiento a la izquierda
>>	Desplazamiento a la derecha

```
b1 = 3 & 2 #11&10=10 (igual a 2)  
b1
```

Tipo lógico o booleano

Una variable tipo lógico o booleano puede tomar sólo puede tomar dos valores:

- True (cierto)
- False (falso)

Para trabajar con ellas están los operadores lógicos:

Operador	descripción
and	Y
or	O
not	No

```
y = True
x = False
r = y and x # si y es True y x False, r es False
r
```

Los valores booleanos son el resultado de expresiones con **operadores relacionales**:

Operador	descripción
==	Igual a
!=	Distinto de
<	Menor que
>	Mayor que
<=	Menor o igual que
>=	Mayor o igual que

```
r = 5 > 3 # r es true
r
```

Tipo Cadena de caracteres

Las cadenas de caracteres son texto encerrado entre comillas simples o dobles.

```
cadena1 = "hola"
cadena2 = 'hola'
```

No existe el tipo de datos caracter. Las cadenas se pueden usar con operadores:

- El operador + concatena cadenas.
- El operador * repite la cadena el número indicado.

Las cadenas son secuencias y como tal se gestionan. Las cadenas se pueden recorrer, indexándolas. El primer índice: 0. Se pueden utilizar intervalos [i : j], i está incluida y j excluida.

a	b	c	d	e
[0]	[1]	[2]	[3]	[4]

Este tipo de dato es inmutable: no se puede sobrescribir. Hay que renombrar. Toda operación sobre cadenas está definida para crear otra nueva como resultado.

```
cadena = "inmutable"  
cadena[0] = "n"
```

```
cadena = "n" + cadena[1:]  
cadena
```

En el paradigma de programación orientada a objetos, se utiliza el concepto de **método**. Los métodos nos permiten realizar determinadas funciones sobre variables, añadiendo después del nombre de la variable un . y el nombre del método a aplicar.

Las cadenas de caracteres poseen métodos específicos para su gestión. Por ejemplo:

- upper() devuelve una cadena en mayúsculas.
- lower() devuelve una cadena en minúsculas.
- swapcase() convierte mayúsculas a minúsculas y viceversa.
- find("subcadena") devuelve un entero que representa la posición donde se inicia la subcadena dentro de otra.
- count("subcadena") devuelve la cantidad de apariciones de la subcadena dentro de otra.
- replace("subcad busqueda", "subcad nueva") reemplaza todas las apariciones de una subcadena por otra especificada en el segundo argumento.
- strip("caracter") elimina caracteres a la izquierda y derecha de una cadena.

A continuación mostramos algunos ejemplos de uso:

```
cad_e = " esta cadena es muy larga "  
cad_e
```

```
len(cad_e)
```

```
cad_e.find("muy")
```

```
CAD = cad_e.upper()  
CAD
```

Entrada y salida

La función `print()` muestra la cadena que encierra entre los paréntesis, omitiendo las comillas que la encierran, en la salida estándar que, normalmente, se corresponde con la pantalla de un ordenador.

Después de escribir la cadena el puntero se sitúa en la siguiente línea de la pantalla. En Python 2 no hace falta paréntesis. También produce como salida el valor de una variable pasada como argumento:

```
nombre = 'Felipe'
print(nombre)
```

Se pueden escribir varias cadenas y variables pasadas todas como argumentos (separadas por comas):

```
print('Hola', nombre, '¿qué tal?')
```

La forma más sencilla de obtener información por parte del usuario es mediante la función `input()`. Esta función toma como parámetro una cadena, para usar como texto a mostrar al usuario, pide la entrada y devuelve una cadena con los caracteres introducidos por el usuario hasta que pulsa la tecla Enter.

La función **`input()`** se utiliza como entrada de datos por teclado a nuestro programa. Es la forma más sencilla de obtener información por parte del usuario. Esta función toma como entrada (se escribe entre paréntesis) una cadena para usar como texto a mostrar al usuario en el que se le solicita un dato y devuelve, como salida (el resultado de usar la función), **una cadena** con los caracteres introducidos por el usuario por teclado hasta que éste pulsa la tecla Enter. Esta cadena se almacena en una variable cuando se le asigna la salida de la función `input()`. Así:

```
nombre = input("Introduce tu nombre: ") # Se llama a la función input() con la cadena de entrada "Introduce tu nombre:".
# La salida de la función input(),
# es decir, lo que escribe el usuario, se asigna a la variable nombre
print("Hola", nombre) # Usamos la función print() para ver el contenido de la variable nombre
```

A continuación mostramos un ejemplo de uso de `print()` y `input()`.

```
#Practica de Entrada/Salida
x = input("Dame un entero: ") # x es una cadena
print("x =", x)
#print("Su cuadrado vale:", x**2) # Da un error
print("Su cuadrado vale:", int(x)**2) # OK
print("Su cuadrado vale (como flotante): {0:6.3f}".format(int(x)**2))
x = int(input("Dame otro entero: ")) # x es un entero
print("Su raíz cuadrada es:", x**0.5)
print("Su raíz cuadrada es: {0:5.3}".format(x**0.5))
print('En un lugar de La Mancha\nde cuyo nombre...')
```

```

print('En un lugar de La Mancha\nde cuyo nombre...')
print('En un lugar', 'de la Mancha', sep=" ", end="...")
print('En un lugar', 'de la Mancha', sep="... ", end="!!!")
x = '''En un lugar
de la Mancha'''
print(x)

```

Otros tipos de datos: Secuencias y Colecciones

En Python tenemos varios tipos de datos para almacenar colecciones de datos o sucesión de elementos.

- **Lista:** Una lista es una secuencia de datos **ordenada**. En otros lenguajes de programación tiene un equivalente en el vector o array (matriz), dependiendo de si es unidimensional o bidimensional. Los elementos de la lista deben ir separados por comas y todo el conjunto entre corchetes. Es una estructura de datos mutable porque además de poder acceder a los elementos, pueden suprimirse, añadirse y modificarse. Se pueden recorrer, indexándolas. Para acceder a un elemento: nombre de la lista[índice del elemento]. El primer elemento tiene índice 0.
- **Tuplas:** La tupla permite agrupar elementos de forma inmutable. Es decir, no permite añadir, ni eliminar, ni modificar sus elementos una vez que se definen. Para declararla se separan los elementos por comas y estos van encerrados entre paréntesis, siendo esto último opcional pero recomendable.
- **Diccionarios:** Los Diccionarios o matrices asociativas son colecciones de parejas donde el primer valor es la clave (keys) y el segundo es el valor asociado a la clave (values). Los pares clave-valor están separados por dos puntos y las parejas por comas y todo el conjunto se encierra entre llaves. El Diccionario es un tipo de datos mutable: Para sumar un nuevo item (clave/valor) basta con especificar la nueva clave y asignarle su valor. Ahora bien, no se puede repetir ninguna clave y es un valor inmutable, podríamos usar números, booleanos, tuplas o cadena pero no listas, mientras que el valor puede ser un número, una cadena, una lista o una tupla.
- **Conjunto:** estructura de datos que nos permite agrupar una colección de datos juntos sin índices ni claves, es decir, sin orden. Los elementos se separan por comas y todo el conjunto se rodea entre llaves {}. Usamos el operador `in` para comprobar que un valor en particular está en el conjunto.

```

listaColores = ['rojo', 'verde', 'azul']
tuplaColoresRGB = ('rojo', 'verde', 'azul')
diccionario={'piso1':'Juan','piso2':'Pepe','piso3':'Luis'}
conjunto_hierbas = {'cilandro', 'perejil', 'hierbabuena'}

```

Funciones

Una función es un fragmento de código con un nombre asociado que realiza una tarea concreta y puede devolver un valor.

Objetivo: Permite reutilizar código.

Las funciones pueden ser:

- Definidas por Python (`input()`, `print()`, `int()`, `float()`, ...) que se encuentran en el módulo `_builtins_`
- Definidas en los módulos/librerías de Python (`sqrt()` en el módulo `math`).
- Definidas por el programador directamente en el código de un programa.

Para poder utilizar una función en un programa se tiene que haber definido antes. Por ello, normalmente las definiciones de las funciones se suelen escribir al principio de los programas.

Para definir una función:

- Se utiliza la palabra clave `def` seguida del nombre de la función más paréntesis de apertura y cierre `( )`. Como toda estructura de control, finaliza con dos puntos `:`.
- Las sentencias que forman el cuerpo de la función (el algoritmo que desarrolla) empiezan en la línea siguiente con **sangría**.
- La primera sentencia del cuerpo de la función puede ser una cadena de texto de documentación de la función (docstring)
- Una función no ejecuta nada hasta que no es llamada.
- Una función puede retornar valores usando `return`
- Las funciones pueden recibir datos de entrada (parámetros)

Ejemplo

Pruebe a realizar el siguiente programa "Cálculo del área de un triángulo".

- Realice un programa que calcule el área de un triángulo. Para ello:
- Defina una función, `area_triangulo`, con dos parámetros de entrada, la base y la altura, y que retorne el área de un triángulo.
- Desde el programa principal, pida al usuario la base y la altura.
- Muestre en pantalla el área calculada.

(Nota): El área del triángulo se calcula con la fórmula $(\text{Base} \times \text{Altura}) / 2$

A continuación le muestro una posible solución.

```
def area_triangulo(base, altura):  
    """Programa para calcular el área de un triángulo"""  
    area = (base*altura)/2.0
```

```

    return (area)

b = float(input("\nbase= "))
a = float(input("\naltura= "))

area = area_triangulo(b,a) #llamada a la función

print("El area del triangulo es ", area)

```

Práctica: Programa para calcular el área del círculo

Realizad el siguiente programa:

- Realice un programa que calcule el área de un círculo Para ello:
- Defina una función, `area_círculo`, con un parámetro de entrada, el radio, y que retorne el área del círculo.
- Desde el programa principal, pida al usuario el valor del radio.
- Muestre en pantalla el área calculada.

(Nota): El área del círculo se calcula con la fórmula $\pi \times \text{radio}^2$

```

# Realizar el programa aquí
# Recordad que pi =3.141592

```

Clases y objetos

La programación orientada a objetos es un paradigma de programación en el cual los conceptos del "mundo real" se modelan a través de objetos, siendo los programas los que regulan las interacciones entre estos objetos a través de sus métodos.

Formalmente, los objetos pueden contener código y datos. Los datos se conocen como atributos o propiedades del objeto y el código se implementa mediante los llamados **métodos**. Los datos de un objeto son modificados a través de la ejecución de un determinado método, el cual puede ser invocado dentro de otro método del mismo objeto o un método de un objeto externo. El valor de los atributos o propiedades del objeto en un momento determinado se conoce como estado del objeto.

Ejemplo Objeto Coche:

- Atributos: marca, modelo, potencia, tipo de carburante,...
- Métodos: arrancar, acelerar, frenar, cambiar de marcha,...

Los objetos se crean a partir de unas plantillas genéricas llamadas clases. Para ello, se crea una instancia de la misma.

Las características fundamentales de este paradigma son:

- **Abstracción:** Se refiere a que un elemento pueda aislarse del resto de elementos y de su contexto para centrar el interés en qué hace y no en cómo lo hace (caja negra).
- **Modularidad:** Es la capacidad de dividir una aplicación en partes más pequeñas independientes y reutilizables.
- **Herencia:** Se refiere a que una clase pueda heredar las características de otra clase superior para obtener objetos similares.
- **Encapsulación:** Consiste en reunir todos los elementos posibles de una entidad al mismo nivel de abstracción para aumentar la cohesión.
- **Polimorfismo:** Se refiere a la posibilidad de identificar de la misma forma comportamientos similares asociados a objetos distintos.

Las clases se definen con la palabra clave `class` seguida del nombre de la clase (la primera en mayúsculas), dos puntos `:` y, a continuación, sangrado, el cuerpo de la clase. Operativa y manejo:

- En el cuerpo se define el método `init` y el resto de métodos:
 - El método `init` (doble barra al principio y al final), llamado constructor, se ejecuta justo después de crear un nuevo objeto a partir de la clase (proceso que denominamos instanciación). Sirve para realizar cualquier proceso de inicialización que sea necesario.
 - El primer parámetro de `init` y del resto de métodos es `self`, el cual sirve para referirse al objeto actual.
- Para crear un objeto hay que escribir el nombre de la clase seguidos de los parámetros necesarios entre paréntesis.
- Se accede a atributos y métodos así: `objeto.atributo` y `objeto.método()`
- En Python todo son clases: números, booleanos, listas,...
- La función `isinstance` nos indica si un objeto concreto es una instancia de una clase particular.
- Python tiene una clase llamada `object`. Cualquier otra clase deriva de ella.

En el siguiente ejemplo se muestra como crear una clase `persona` con sus atributos `edad` y `nombre` a la cual se le añade un método `cumpleaños` que incrementa su edad en 1. De esta forma no será necesario modificar el valor del atributo directamente.

```
class Persona():
    def __init__(self, edad, nombre): # método constructor de objeto.
        self.edad = edad # pone la edad
        self.nombre = nombre # pone el nombre

    def cumpleaños(self): # método cumpleaños, incrementa la edad en 1
        self.edad += 1

luis = Persona(31, "Luis")
```

```
print("Nombre: ",luis.nombre," Edad: ",luis.edad)
luis.cumpleaños()
print("Edad despues del cumpleaños:", luis.edad)
```

Estructura lógica de un programa y control de flujo.

Hay ciertos patrones conceptuales que se usan para estructurar los programas. Esas estructuras no son exclusivas de Python, sino que forman parte de cualquier lenguaje de programación. Ya hemos hablado de algunos de estos patrones:

- **Entrada:** Obtener datos para el programa desde el exterior. Puede consistir en leer datos desde un fichero, o incluso desde algún tipo de sensor o desde una base de datos. En nuestro caso, hasta ahora, las entradas han venido dadas por usuario que ha introducido los datos a través del teclado.
- **Salida:** Mostrar los resultados del programa al exterior. Por ejemplo, se pueden almacenar en un fichero o enviarlos a un dispositivo como un controlador, un altavoz, ... en nuestro caso, hasta ahora, se han mostrado en la pantalla.
- **Reutilización:** Escribir un conjunto de instrucciones una vez, darles un nombre y así poder reutilizarlas luego cuando se necesiten en cualquier punto del programa: hemos aprendido el uso de las funciones y de los métodos.

El resto de nuestro programa va a consistir en sentencias a ejecutar. La ejecución puede ser:

- **Ejecución secuencial:** Se ejecuta una sentencia tras otra en el mismo orden en que se van encontrando en el programa o script.
- **Ejecución condicional:** Se comprueba ciertas condiciones y luego, de acuerdo con el resultado, se ejecuta u omite una secuencia de sentencias.
- **Ejecución repetida:** Se ejecuta un conjunto de sentencias varias veces, normalmente con algún tipo de variación.

Lo normal a la hora de ejecutar un programa es que la ejecución sea secuencial: las instrucciones se van ejecutando una a una, de arriba a bajo. Ahora bien, utilizando determinadas estructuras de control, este tipo de ejecución puede variar, convirtiéndose en condicional (usando sentencias condicionales) o en repetida (usando bucles). A continuación explicaremos dichas sentencias.

Las estructuras de control de flujo pueden ser:

- **Sentencias condicionales o de selección:**
 - if
 - if ... else
 - if ... elif ... elif ... else
- **Bucles o sentencias iterativas:**

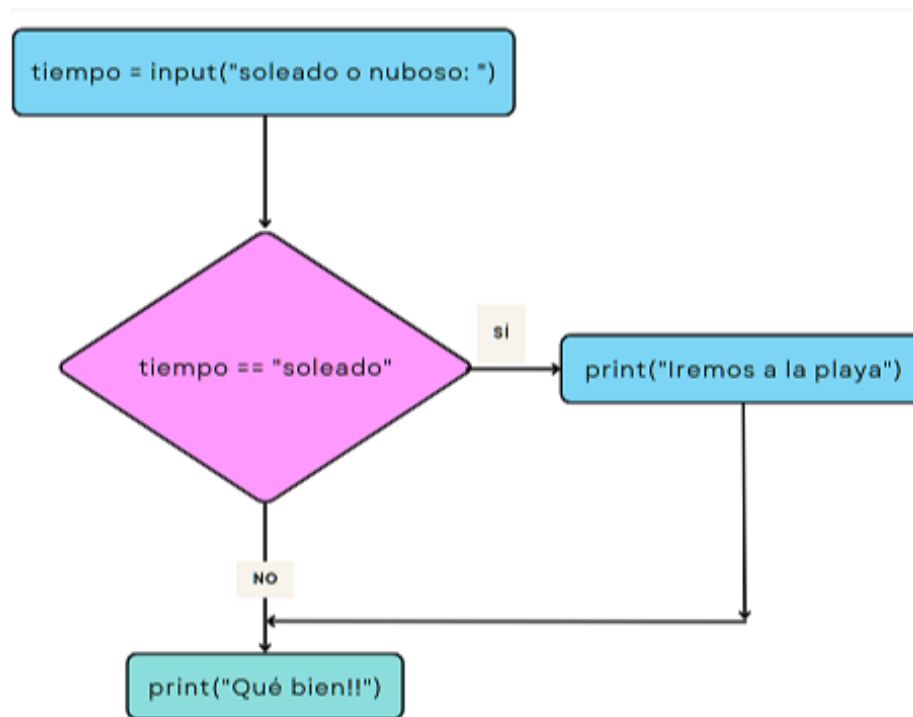
- while
- for ... in

Sentencias condicionales

Permiten ejecutar un fragmento de código u otro dependiendo de que se cumpla una condición.

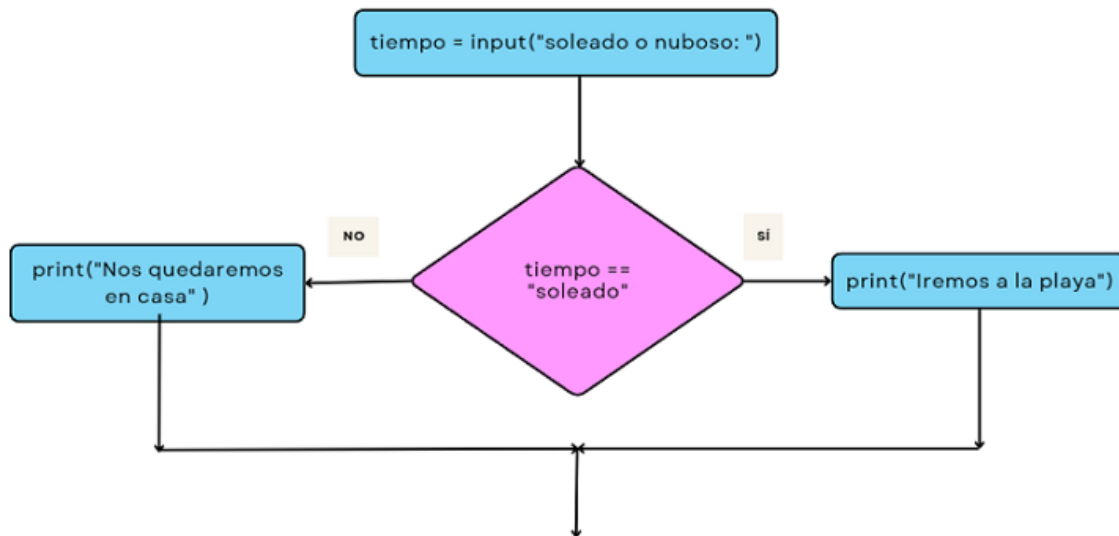
La forma más simple se construye mediante *if* seguido de la condición a evaluar (tipo de dato bool) , dos puntos (:), y en la siguiente línea y sangrado, el código a ejecutar en el caso de que se cumpla la condición. Recordemos que la condición es una expresión booleana es decir, el resultado debe ser True o False. Si la condición es verdadera, la sentencia o sentencias sangradas se ejecutarán. Si la condición es falsa, la sentencia o sentencias sangradas serán omitidas. Para la condición se utilizan los operadores relacionales que estudiamos en el tema anterior (`==`, `!=`, `>`, `>=`, `<` y `<=`).

A continuación presentamos un ejemplo de uso que se corresponde con este algoritmo:



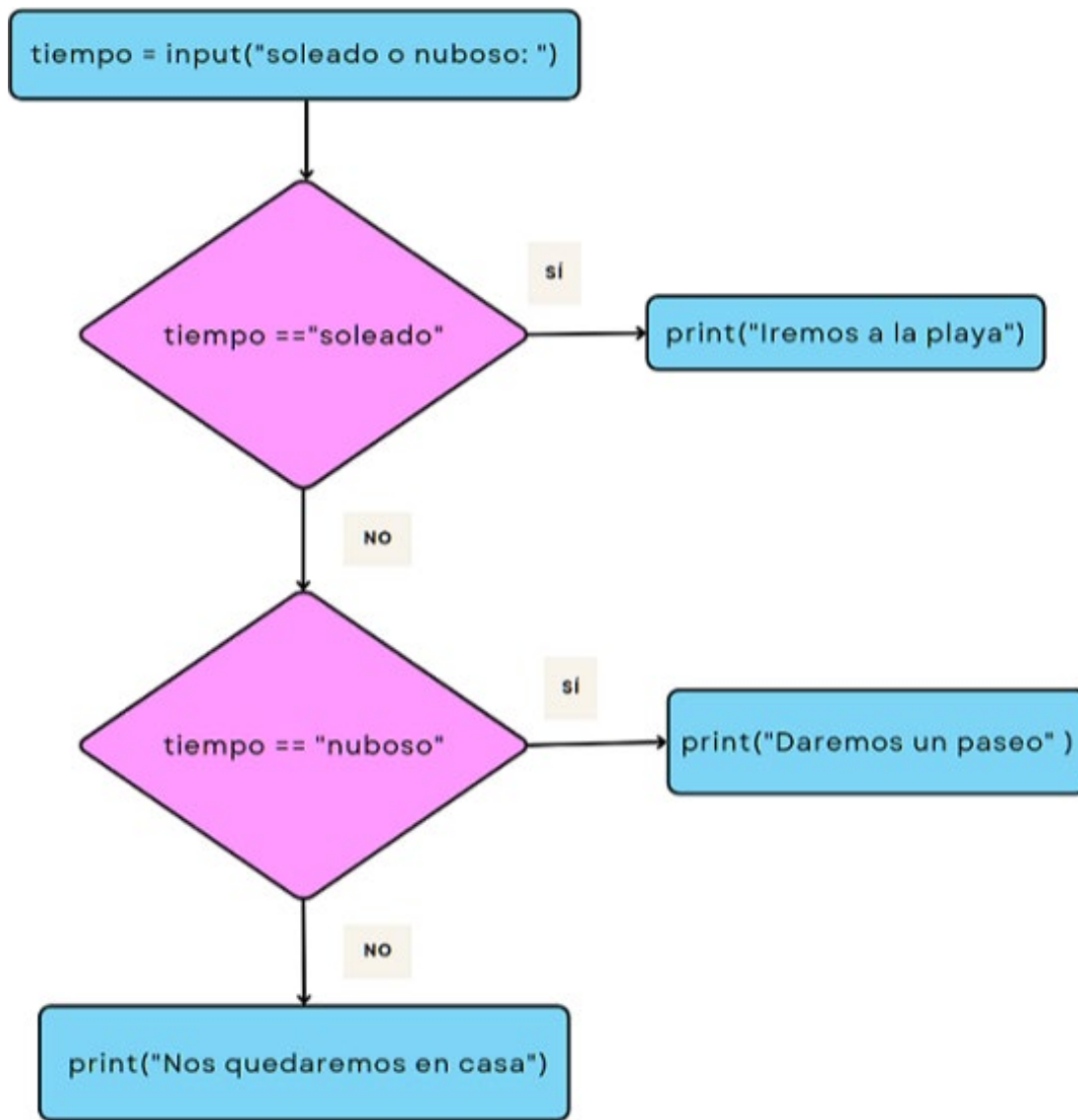
```
tiempo = input("soleado o nublado: ")
if tiempo == "soleado":
    print("Iremos a la playa")
print("Qué bien!!")
```

Si queremos que en caso de que no se cumpla la condición se ejecute otro bloque de código usaremos la construcción `if ... else`. A continuación vemos el algoritmo y su programa asociado:



```
tiempo = input("soleado o nublado: ")
if tiempo == "soleado":
    print("Iremos a la playa")
else:
    print("Nos quedaremos en casa")
```

Se pueden anidar condiciones. Para ello usaremos la construcción `if ... elif ... else`. `elif` es una abreviatura de `else if`. A continuación vemos el algoritmo correspondiente y su ejemplo asociado:



```
tiempo = input("soleado o nublado: ")
if tiempo == "soleado":
    print("Iremos a la playa")
elif tiempo == "nublado":
    print("Daremos un paseo")
else:
    print("Nos quedaremos en casa")
```

Ejemplo

Determinar si un número es divisible por 2, 3 ó 5

```
num = int(input("Introduce un numero: ")) # Recuerde que input() devuelve
una cadena de caracteres, de ahí la conversión de tipo
```

```
if num % 2 == 0:
    s da el resto de la división
```

Recuerde que el operador % no

```

    print("Es divisible por 2")
elif num % 3 == 0:
    print("Es divisible por 3")
elif num % 5 == 0:
    print("Es divisible por 5")
else:
    print("No es divisible por 2, 3 ó 5")

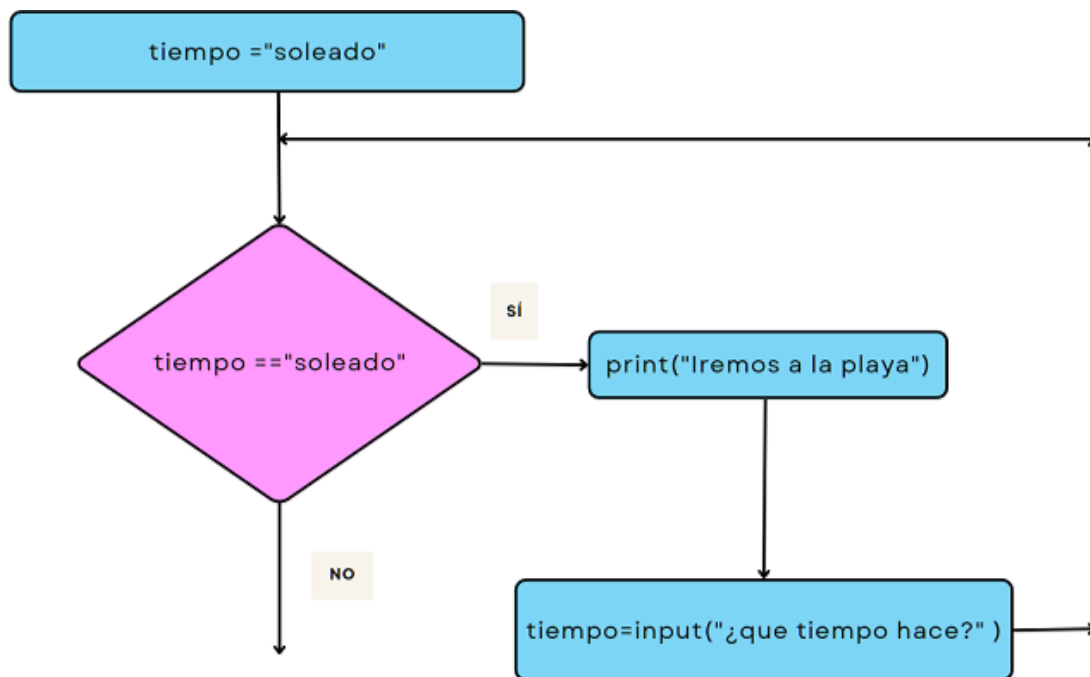
```

Bucles o sentencias iterativas

Los bucles o sentencias iterativas nos permiten ejecutar un fragmento de código un cierto número de veces, mientras se cumpla una determinada condición.

Un tipo de estructura de bucle es `while` seguido de la condición a evaluar, dos puntos (:), y en la siguiente línea y sangrado, el código a ejecutar en el caso de que se cumpla la condición. Para que el bucle no sea infinito, nos debemos asegurar la salida del bucle, es decir, que la condición se convierta en `False`. Si debemos salir de un **bucle infinito** se teclaa `Ctrl + C`.

A continuación, continuaremos con nuestro ejemplo mostrando el algoritmo y el programa asociado:



```

tiempo = "soleado"
while tiempo == "soleado":    # Si la condición es cierta se ejecutan la
    print("Iremos a la playa") # s sentencias dentro del bucle y se vuelve a testear
    tiempo = input("¿Qué tiempo hace? ") # Sentencia dentro del bucle que
                                         # permite que la condición a testear pueda variar

```

Otra forma de construir un bucle es con la estructura `for ... in`. Se utiliza para recorrer los elementos de un lista.

- La construcción utiliza una variable que en cada ciclo toma el valor de un nuevo elemento de la lista de la cláusula `in`.
- Esta variable que usamos en la construcción se declara en *tiempo de ejecución* (dinámicamente).
- El bucle terminará cuando se alcance el último elemento de la lista o cuando se fuerce su fin con `break`.
- Si se necesita iterar sobre una secuencia de números, es apropiado utilizar la función `range()`:
 - `range(x)`, recorre el valor de 0 a x-1.
 - `range(x, y)`, recorre desde x hasta y-1.
 - `range(x,y,s)`, recorre desde x hasta y-1, de s en s.

```
# Suma los números del 1 al 10
suma = 0
```

```
for i in range(1, 11):    # Recorre los números del 1 al 10
    suma += i             # Acumula cada número en la variable 'suma'
```

```
print("La suma de los números del 1 al 10 es:", suma)
```

Errores y Excepciones

Los **errores** de sintaxis o errores de interpretación son los más comunes. Ocurren cuando no se ha introducido correctamente una sentencia o línea de programa. Esto se detecta antes de que comience el programa por el intérprete de Python, lanzando un mensaje como los siguientes:

```
aa="hola"+3.0    ##tipos incompatibles para la operación suma

while True
    print ("hola")
```

Las **excepciones** son errores detectados durante la ejecución del programa aunque las sentencias o expresiones sean sintácticamente correctas. No tienen que ser fatales y se pueden tratar mediante una estructura *try and except*.

```
while True:        #bucle infinito
    try:
        numerador=float(input("Introduzca el numerador: "))
        denominador=float(input("Introduzca el denominador: "))
        división=numerador/denominador
        print ("El resultado de la división es: {0:.3f}".format(división))
    #Toma 3 decimales máximo
    break          #Si hemos llegado aquí, el programa ha funcionado correct
amente y salimos de la estructura "try"
```

```

except ValueError:
    print ("No ha introducido un número. Pruebe de nuevo...\n")
except ZeroDivisionError:
    print ("No se puede dividir por 0. Pruebe de nuevo...\n")
print ("\nPrograma finalizado. \nHasta la próxima")

```

Ejemplo

Vamos a realizar un programa que sume los n primeros números con saltos de s números, empezando por el número m . Para ello desde el programa principal:

- Pide m , n y s
- Comprobar y tratar con excepciones cuando proceda que m , n y s sean números enteros positivos diferentes de 0.
- Si salta excepción se reinicia el programa.
- Si los datos son correctos, mostrar la suma por pantalla.

```

print ("Programa para sumar los n primeros números con saltos de s número s\n")
while True:      #bucle infinito
    try:
        m=int(input("Introduzca m: "))
        n=int(input("Introduzca n: "))
        s=int(input("Introduzca s: "))
        if ((m<=0) or (n<=0) or (s<=0)):
            print ("Datos incorrectos... pruebe de nuevo")
        else:
            suma =0
            for i in range(m,n+1,s):
                suma=suma+i
            print ("El resultado es: ",suma)
            break      #Salimos...
    except ValueError:
        print ("Debe introducir un número entero positivo. Pruebe de nuevo")
print ("\nPrograma finalizado. \nHasta la próxima")

```

Práctica: Programa que determina si una contraseña es segura o no

Debe realizar un programa que determine si una contraseña es segura o no. Para ello, desde el programa principal:

- Pide una contraseña al usuario.
- Llama a una función que determina si la contraseña es segura o no.
- Informa al usuario si la contraseña es segura o no.
- Si la contraseña no es segura, se pide otra contraseña. No se abandona el programa hasta que la contraseña seleccionada no sea segura.

La función debe realizar las siguientes acciones:

- Evaluará si la contraseña pasada como único parámetro es segura o no. Para ello chequeará si:
- Tiene al menos 8 caracteres.
- Contiene, al menos, una letra mayúscula.
- Contiene, al menos, una letra minúscula
- Contiene, al menos, un número.
- La función deberá devolver un valor booleano al programa principal para informarle si la contraseña que se le ha pasado como único parámetro es segura o no.

```
# Realizar el programa aquí  
#
```

Bibliografía

Manuales y Tutoriales:

- [documentacion de la Página oficial](#)
- [Más documentación](#)
- Tutorial oficial de Python 3 escrito por Guido van Rossum en distintos formatos:

[Siempre actualizado y en su versión en línea](#)

[En formato pdf, traducción al castellano de una versión más antigua](#)

Libros:

- [Introducción a la Programación con Python 3](#). Andrés Marzal Varó, Isabel Gracia Luengo y Pedro García Sevilla. Universitat Jaume I, 2014.

Shell on-line de Python: Permite probar código Python desde el navegador web sin necesidad de instalar nada en el sistema:

- <https://www.python.org/shell/>

Autoría

La información contenida en este Notebook ha sido elaborada por los profesores:

- Carolina Mañoso Hierro. email: carolina@scc.uned.es
- Miguel Romero Hortelano. email: mromero@scc.uned.es

UNED

ETS de
Ingeniería
Informática